

VANFIST: Versatile Autonomous Navigation for Flexible Industrial and Strategic Tasks—Design and Edge AI Perception

Minh Phuc Nguyen

*Department of Electrical, Computer, and Biomedical Engineering
Union College
Schenectady, NY, USA
nguyenp@union.edu*

Abstract—This paper describes the hardware and perception stack of VANFIST, a custom 1:5 scale autonomous ground vehicle. VANFIST is built around a welded steel chassis, and uses an ESP32-S3 actuation controller with an NVIDIA Jetson Orin Nano as main computing brain. To adapt perception to its low camera viewpoint, we collected and labeled 304 full-HD images for binary drivable-area segmentation. Increasing the first-stage stride of the fine-tuned SegFormer-B0 raised compute-only throughput from 6.35 to 57.96 FPS. On the Jetson, the deployed TensorRT pipeline processed all 3,749 frames of a full-HD video without skipping at a median 41.56 FPS. Because the video was not ground-truth labeled, these results establish deployment throughput and provide qualitative evidence rather than segmentation accuracy.

Index Terms—Autonomous Ground Vehicles, Edge AI, Semantic Segmentation, SegFormer, TensorRT.

I. INTRODUCTION

Navigating a small autonomous vehicle through a college campus is surprisingly difficult. While educational platforms such as TurtleBot3 and NVIDIA JetBot are effective for controlled indoor experimentation, their outdoor use is constrained by many factors like small-obstacle traversal, uneven surfaces, and variable illumination [1], [2]. To solve these problems, we started the VANFIST project. Our goal was to build and program a 1:5 scale autonomous ground vehicle that could handle real outdoor paths, avoid obstacles, and drive itself using edge computing.

During the previous terms (Winter and Spring 2025), we focused on the basics: testing vehicle movement, altering the chassis, and trying an off-the-shelf YolactEdge model on the Jetson [3]. At first, we used KITTI-360 training data to fine-tune the model. However, KITTI-360 shows urban roads from a full-size survey vehicle [4], whereas our camera sits only 48 cm above a campus walkway. The retained project does not contain a ground-truth evaluation of that earlier model, so we describe the outcome only as we observed it: its masks did not identify the campus drivable surface reliably enough to use. This term, we changed our approach. We focused on building a custom dataset, training a better vision model, and integrating the software with the physical body of the vehicle. Our specific work includes:

- 1) **Hardware Build:** Tearing down a 1:5 scale chassis, welding a custom steel subframe, and wiring an ESP32-S3 microcontroller to talk to the NVIDIA Jetson Orin Nano.
- 2) **Custom Vision Dataset:** Taking 304 ground-level photos on campus, labeling drivable pavement in CVAT [5], and investigating synthetic nighttime images for training.
- 3) **Fast Edge AI:** Training a SegFormer vision model [6] and speeding it up with NVIDIA TensorRT [7] so it runs smoothly on the Jetson.

II. SYSTEM ARCHITECTURE AND HARDWARE PLATFORM

A. Mechanical Chassis and Structural Redesign

The mechanical base of VANFIST originated from a modified commercial ride-on electric vehicle chassis. To ensure structural rigidity under dynamic loads and provide robust mounting for heavy battery modules and sensor brackets, the stock plastic structural elements and drive mechanics were stripped. A customized rectangular steel tube chassis was designed, welded, and lathe-machined. Fig. 1 shows the completed chassis and mounted hardware.

B. Embedded Computing and Power Architecture

The electronic and compute architecture is partitioned into two functional tiers: high-level cognitive computing and low-level deterministic motor actuation.

- **High-Level Compute Tier:** The central computing unit is an NVIDIA Jetson Orin Nano Developer Kit [8], featuring a GPU with 8 GB of RAM memory and delivering up to 40 TOPS of INT8 compute capability. The Jetson executes the ROS 2 Humble middleware, deep learning inference engines, LiDAR point cloud processing, and global pathfinding nodes.
- **Low-Level Actuation Tier:** Actuation is managed by an ESP32-S3 microcontroller. The ESP32-S3 drives GPIO-isolated H-bridge motor driver circuits to command forward, reverse, left, right, and dynamic braking operations. A robust USB-to-UART serial protocol connects the Jetson Orin Nano to the ESP32-S3, enabling bidirectional communication: receiving velocity twist commands ($\mathbf{v}, \boldsymbol{\omega}$)



Fig. 1: Fabricated 1:5 scale VANFIST testbed featuring custom welded steel subframe, NVIDIA Jetson Orin Nano edge computer, ESP32-S3 actuation controller, 2D planar LiDAR, and isolated DC-DC power architecture.

from the high-level planner and transmitting low-level hardware telemetry.

- **Power Distribution Subsystem:** Power is supplied by a primary 12V high-discharge lead-acid/lithium battery pack. High-efficiency DC-DC buck converters provide isolated, regulated 5V/4A rails for the Jetson Orin Nano and sensor peripherals, preventing voltage sag and electrical inductive back-EMF spikes during high-torque motor transients.

C. Sensor Suite

The sensing payload includes:

- 1) **Monocular Vision:** A high-definition 1920×1080 USB camera mounted at an optical height of $h_c = 48$ cm above ground level, providing an optimal perspective of near-field road surfaces.
- 2) **2D Range LiDAR:** A 360° planar LiDAR scanning at 10 Hz with an effective range of 12 m, utilized for local obstacle costmap generation and boundary detection.
- 3) **GPS Receiver:** A high-sensitivity GPS module providing latitude/longitude fixes at 5 Hz for global topological waypoint localization.

III. CAMPUS MAPPING AND WAYPOINT GRAPH

The primary transit axis between *Messa House* and *Schaffer Library* on the Union College campus was selected for the reference route (Fig. 2). This path was chosen because it represents a critical pedestrian and service path featuring wide paved surfaces and multiple pathway intersections connecting residential halls, dining centers, and academic libraries.

We built the campus route by using Google My Maps [9] to mark GPS waypoints along sidewalks and other drivable paths. On long, straight sections, the waypoints were spaced about 2.0 meters apart because the vehicle needed fewer points to follow the path. Around corners and curved sections, we reduced the spacing to approximately 0.5 meters to give the

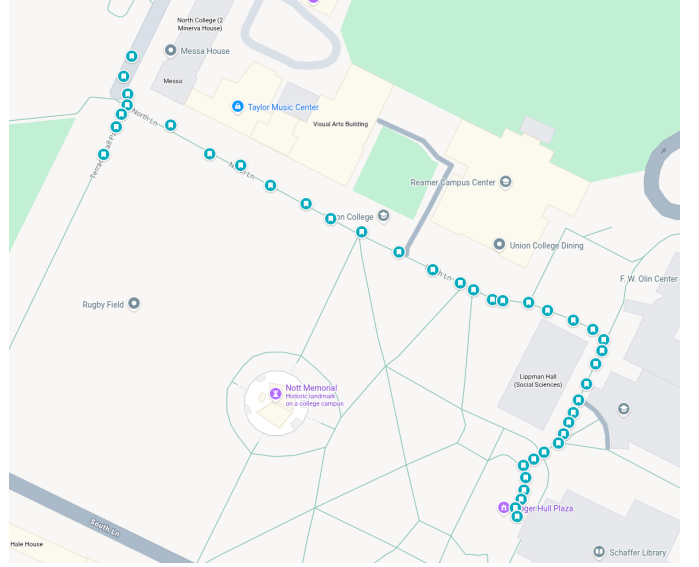


Fig. 2: Mapped campus transit path between Messa House and Schaffer Library on the Union College campus, establishing the reference GPS waypoint trajectory.

vehicle more detailed steering guidance. The closer spacing helps the vehicle follow turns without cutting across curbs or leaving the intended path, while the wider spacing on straight sections keeps the route map simple and avoids unnecessary computation.

IV. VISION PERCEPTION AND EDGE SEMANTIC SEGMENTATION

A. Why a Binary Segmentation Model

As we learnt from past experiences, we decide to narrow the task. We didn't try to fine-tune a complicated model with multiple class segmentation. Instead, we ask a useful question: which area belong to pavement the vehicle is allowed to enter? We therefore kept one foreground annotation, *Drivable*, and treated every unlabeled pixel as background. We chose SegFormer-B0 [6] because its hierarchical encoder preserves multi-scale context while its small decoder is realistic for edge deployment. In hindsight, beginning with the B0 variant was also a useful restraint. With only a few hundred labeled frames, a larger backbone would have added compute and overfitting risk before fixing the more basic data problems.

The retained dataset contains 304 1920×1080 frames. In CVAT [5], polygons marked the drivable region; the exported RGB masks were then converted to class-index PNGs with 0 for background and 1 for drivable. Drivable pavement accounts for 35.39% of labeled pixels. We only captured data in daylight. We explored nighttime generation with Nano Banana Pro [10] and a separate weather-transfer pipeline, but the examples were inconsistent and expensive to scale. We therefore abandoned the synthetic data.



(a) Camera input.



(b) TensorRT segmentation overlay.

Fig. 3: Temporally central frame (frame 1,875 of 3,749) from the deployment video. The video has no ground-truth masks; this comparison is qualitative only.

B. Training and Deployment Video Evaluation

We trained SegFormer-B0 for binary drivable-area segmentation using 304 manually labeled 1920×1080 images. The deployed TensorRT model was tested on a 3,749-frame, 1920×1080 , 30-FPS video. The complete pipeline processed every frame without skipping and achieved a median throughput of 41.56 FPS over three runs. The video was not ground-truth labeled; therefore, this test measures deployment throughput and provides qualitative results only. Fig. 3 shows the camera input and segmentation overlay for the temporally central video frame.

C. From a 6 FPS Model to a 40 FPS Pipeline

The trained PyTorch model was exported to ONNX and compiled for the Jetson Orin Nano Super with TensorRT [7]. Our first instinct was to optimize the surrounding video code: persistent buffers, pinned transfers, double buffering, and faster decode. Profiling forced us to change direction. Three new compute-only repetitions of the original engine produced a median 6.35 FPS and 157.51 ms of GPU compute. No decoder or queue design can push a pipeline beyond a model that slow.

The first encoder stage was the bottleneck. At stride 4, it creates $270 \times 480 = 129,600$ queries attending to 1,980 spatially reduced key/value tokens, and the execution context required about 1.268 GiB. Increasing only the first learned patch-projection stride to 8 reduces the first-stage spatial area by four and the retained context to roughly 132 MiB. Builder optimization level 5 explored tactics requiring impractical memory and was stopped; level 3 completed the engine in 335.96 s.

The deployed engine still accepts $1 \times 3 \times 1080 \times 1920$ FP16 input, but its logits change from $1 \times 2 \times 270 \times 480$ to $1 \times 2 \times 135 \times 240$. A fused CUDA kernel performs argmax and expands the mask for the full-HD overlay. The remainder of the pipeline uses a persistent TensorRT context and CUDA stream, reusable pinned/device buffers, fused preprocessing and overlay kernels. The repeated compute-only and full-pipeline measurements are reported separately in Table I.

TABLE I: Jetson Orin Nano Super Model and Video-Processing Performance

Configuration	Measured operation	Median throughput	Processing time
Original model	Inference only	6.35 FPS	157.51 ms/frame
Optimized model	Inference only	57.96 FPS	17.25 ms/frame
Optimized model	Complete video processing (3,749 frames)	41.56 FPS	90.22 s total

The original and optimized models use first-stage strides of 4 and 8, respectively. Complete video processing includes decoding, preprocessing, inference, overlay generation, and encoding. Values are medians of three runs.

V. CONCLUSION AND FUTURE WORK

VANFIST demonstrates that a SegFormer-B0 model for binary drivable-area segmentation can be deployed on the Jetson Orin Nano Super for real-time video processing. The optimized TensorRT pipeline processed all 3,749 frames of the test video at a median 41.56 FPS, exceeding the source rate of 30 FPS. Future work will reserve an independently collected and labeled test set that remains excluded from training and model selection. It will include independent routes recorded at night, after rain, and under shadows, and will evaluate mIoU, boundary quality, and false-drivable errors.

REFERENCES

- [1] ROBOTIS, “TurtleBot3 Features,” <https://emanual.robotis.com/docs/en/platform/turtlebot3/features/>, accessed: Aug. 27, 2026.
- [2] C. A. García-Pintos, N. G. Aldana-Murillo, E. Ovalle-Magallanes, and E. Martínez, “A deep learning-based visual map generation for mobile robot navigation,” *Eng.*, vol. 4, no. 2, pp. 1616–1634, 2023.
- [3] H. Liu, R. A. R. Soto, F. Xiao, and Y. J. Lee, “YolactEdge: Real-time instance segmentation on the edge,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 9579–9585.
- [4] Y. Liao, J. Xie, and A. Geiger, “KITTI-360: A novel dataset and benchmarks for urban scene understanding in 2D and 3D,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 3, pp. 3292–3310, 2023.
- [5] CVAT.ai Corporation, “CVAT: Computer Vision Annotation Tool,” <https://www.cvat.ai>, 2024, accessed: Jun. 2026.

- [6] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, "SegFormer: Simple and efficient design for semantic segmentation with transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 12 077–12 090.
- [7] NVIDIA Corporation, "NVIDIA TensorRT Deep Learning Inference SDK," <https://developer.nvidia.com/tensorrt>, 2024, accessed: Jun. 2026.
- [8] —, "Jetson Orin Nano Developer Kit User Guide," <https://developer.nvidia.com/embedded/learn/jetson-orin-nano-devkit-user-guide>, 2024, accessed: Jun. 2026.
- [9] Google LLC, "Google My Maps Geographic Data Platform," <https://www.google.com/maps/about/mymaps>, 2026, accessed: Jun. 2026.
- [10] Google DeepMind, "Introducing Nano Banana Pro," <https://blog.google/innovation-and-ai/products/nano-banana-pro/>, Nov. 2025, accessed: Aug. 27, 2026.